

Technical Architecture Specification: Enterprise AI-Powered Personal Journal

Document Reference: ARCH-AI-2026-V1 | **Audience:** Solution Architecture Forum | **Status:** Approved Baseline Blueprint

1. Executive Summary & Architectural Vision

Modern cloud ecosystems, managed serverless compute, and turnkey foundation model APIs have radically simplified the development and deployment of generative AI solutions. Where multi-turn conversational agents previously required specialized machine learning infrastructure, GPU provisioning, and custom vector databases, enterprise-grade generative applications can now be constructed rapidly using battle-tested web standards, declarative security rules, and managed APIs.

This technical architecture specification reconciles and elevates two reference implementations—a modern TanStack Start/Postgres system and a Next.js App Router/Serverless architecture—into a definitive target architecture. The resulting application is an enterprise-ready, user-authenticated Personal Journal leveraging **Firestore Authentication**, **Google Cloud Firestore**, and the **Google Gemini API**. This architecture guarantees strict multi-tenant isolation, comprehensive prompt sanitization, automated distress detection, and aggressive token-minimization controls that keep steady-state operating expenses well below \$50 to \$100 per month.

2. Comparative Analysis of Baseline Implementations

The following comparative analysis contrasts the architectural choices of the two reference architectures against the unified enterprise target pattern proposed for technical leadership review:

Architectural Dimension	Implementation A (Stack1)	Implementation B (Stack2)	Target Enterprise Blueprint
Framework & Runtime	TanStack Start (React 19 + Vite 7), Tailwind CSS v4	Next.js App Router (React), Tailwind CSS	Next.js App Router or React 19 SSR containerized on Cloud Run
Identity & Auth Engine	Supabase Auth (JWT session management)	Better Auth (Server-side cookies, secret hashing)	Firestore Authentication (Google Identity / OAuth + Passwordless)
Primary Data Store	PostgreSQL (Supabase instance)	Neon Serverless PostgreSQL with Drizzle ORM	Google Cloud Firestore (Hierarchical User-Rooted NoSQL)

Architectural Dimension	Implementation A (Stack1)	Implementation B (Stack2)	Target Enterprise Blueprint
Multi-Tenant Isolation	Database Row-Level Security (RLS) policies on auth.uid()	Application-level parameterized queries by verified session UID	Declarative Firestore Security Rules (request.auth.uid == userId)
AI Model Orchestration	App2 AI SDK; Gemini 3.7 Flash server-side with BYOK fallback	AI SDK Mock adapter for deterministic, zero-cost CI testing	Server-side Gemini 2.5/3.7 Flash with Mock SDK for testing
Token Control Policy	Sliding window: 3 past entries + ≤300-word summary; 300 token max	Bounded prompts; rate limits and token ceilings required	Enforced 3-entry sliding window, rolling summaries, 300-token caps
Responsible AI Layer	safety.ts crisis keyword detection, non-diagnostic response routing	OWASP LLM Top 10 mitigation, prompt size bounds, generic errors	Dual-stage moderation: Regex crisis triage + input sanitization
Hosting Infrastructure	Edge Server Functions / Google Cloud Run	Containerized Cloud Run with Secret Manager & Load Balancer	Google Cloud Run (Scale-to-Zero Container, Auto-Provisioned TLS)

3. Target System Architecture & Data Model

The architecture relies on a strictly bounded serverless pipeline. Client runtimes communicate exclusively with authenticated HTTPS endpoints. The Gemini API keys and database admin privileges never cross into browser-executed JavaScript.

3.1 End-to-End Component Flow

The operational flow proceeds in four distinct security zones:

1. **Presentation Layer:** React frontend providing the user dashboard, history navigation, and the Reflect Drawer. All journal filtering (date, keyword, mood tags) executes client-side across user-scoped records.
2. **Identity & Gateway Layer:** Firebase Authentication validates tokens on every API route and server action. Requests lacking a cryptographically sound bearer token are rejected before hitting downstream resources.
3. **Persistence Layer:** Cloud Firestore stores user settings, entry histories, and multi-turn message exchanges organized by explicit document ownership paths.
4. **AI Orchestration Gateway:** The server-side AI runner sanitizes context, executes distress detection, caps token budgets, dispatches inference calls to Google Gemini, and logs anonymized telemetry.

3.2 Firestore User-Rooted Schema

Relational models introduce operational overhead when configuring join-based tenant isolation. Firestore resolves this by nesting journal data strictly beneath user identifiers:

```
/users/{userId}
├── /profile/settings
│   engine: "gemini-flash"
│   rolling_summary: "User focused on quarterly software engineering goals..."
│   total_prompt_tokens: 142050
│   total_completion_tokens: 28400
│   created_at: Timestamp
└── /entries/{entryId}
    title: "Reflections on System Scalability"
    content: "Today we reviewed container autoscaling limits..."
    mood: "productive"
    tags: ["architecture", "cloud", "career"]
    created_at: Timestamp
    updated_at: Timestamp
    conversation: [
      { role: "user", text: "How can I summarize my trade-offs?", timestamp: Timestamp },
      { role: "model", text: "Here is a 3-point framework for your review...", timestamp: Timestamp }
    ]
```

3.3 Firestore Declarative Security Rules

Data isolation is enforced natively at the database storage engine. Even if client-side code is modified or compromised, cross-tenant data exfiltration is physically blocked by the storage engine:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Explicit tenant isolation rule
    match /users/{userId}/{document=**} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId;
    }

    // Explicit global deny for any unmapped paths
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

4. Detailed End-to-End User Flow Execution

The application guides users through six discrete execution stages, each designed with deterministic validation and privacy guards:

Flow Stage	User & Client Action	Server & Infrastructure Response	Security & Governance Enforcement
1. Landing & Auth	Arrives at landing; executes Google or Passwordless Sign-In.	Firebase Auth issues cryptographically signed JWT bearer token.	Landing route tagged with noindex; auth cookies use strict SameSite.
2. Dashboard Gate	Client navigates to protected /dashboard workspace.	Server middleware verifies token via Firebase Admin SDK; hydrates user state.	Unauthenticated requests redirected immediately to /auth.
3. Journal Drafting	User writes reflections; selects mood chip and enters custom tags.	Client opens multi-turn Reflect Drawer; binds draft to active session.	Payload schema validated against input limits (e.g., max 5,000 chars).
4. AI Synthesis	User triggers conversational prompt or reflection analysis.	Server constructs sliding context; calls Gemini Flash API with 300 token cap.	Crisis detection runs pre-inference; agent tools disabled in emergency state.
5. Persistence	UI optimistically displays the generated AI insight.	Entry content, AI response, and token count written to Firestore.	Firestore Security Rules verify auth.uid == userId; cross-user write denied.
6. History & Search	User navigates calendar, filters by mood chips, or searches text.	Client filters in-memory records already retrieved within the user scope.	Zero backend search query surface; prevents injection and data leakage.

5. Context Construction & Token Optimization Strategy

Unbounded conversational contexts represent the single greatest cost and latency risk in production generative AI systems. Without disciplined pruning, conversational histories balloon rapidly, causing exponential token consumption and degradation of model focus.

5.1 The Sliding Window & Rolling Summary Algorithm

The platform implements a strict dual-tier context bounding algorithm:

- **Sliding History Window:** Only the current draft plus the **3 most recent chronological journal entries** are included in the prompt payload. Older entries are pruned automatically.
- **Rolling Summary Buffer:** Long-term behavioral context is preserved using a condensed, asynchronous rolling summary capped strictly at **300 words** (~400 tokens), updated in the

background upon every 5th entry.

- **Hard Output Ceiling:** Every LLM completion request enforces a hard ceiling of **300 output tokens**, ensuring punchy, empathetic, and actionable replies while eliminating runaway generation loops.
- **Deterministic Budget Caps:** Maximum cumulative input per turn is bounded at ~2,000 tokens, guaranteeing that every conversational exchange remains predictable in cost.

6. Security, Threat Modeling & Responsible AI Guardrails

The application adheres to the OWASP Top 10 for Large Language Model Applications and Google's Secure AI Framework (SAIF):

OWASP Risk / Threat	Vulnerability Scenario	Architecture Mitigation Control
LLM01: Prompt Injection	Adversarial journal entry attempting to override system instructions or exfiltrate system prompt.	Strict structural isolation between System Instructions and User Content blocks; automated CI testing via Promptfoo/Garak.
LLM02: Sensitive Data Leak	Cross-tenant retrieval of journal reflections belonging to another authenticated user.	Physical path scoping: Firestore rules match <code>/users/{userId}</code> strictly to verified JWT UID; client input ID never trusted.
LLM04: Denial of Wallet	Malicious loop generating massive prompt batches to exhaust API credits or cloud compute.	Server-side rate limits (max 10 turns/hour), input length capping ($\leq 5,000$ chars), and hard 300-token completion limits.
LLM08: Excessive Agency	AI companion autonomously publishing user journal entries to external public networks.	Strict draft-only tooling; no external publishing APIs exist without explicit multi-step user authorization and confirmation.
Responsible AI: Crisis Safety	User expressing severe depression, self-harm intentions, or acute emotional crisis.	Pre-inference regex filter intercepts trigger words; bypasses generative engine to render vetted supportive helpline resources.

7. Cost Modeling & Financial Projections

The following projections illustrate the operational costs for 200 vs. 500 Monthly Active Users (MAU), operating under the token-optimized architecture:

Infrastructure Component	200 MAU Workload (~4,800 turns)	500 MAU Workload (~5,000 turns)	Cost Engineering Strategy
Compute (Cloud Run)	\$5 – \$25 / month	\$25 – \$70 / month	Scale-to-zero compute; minimum 2 instances only when high availability is required.
Database & Storage (Firestore)	\$0 – \$10 / month (within free tier)	\$10 – \$25 / month	Indexed reads; client-side caching prevents redundant fetches.
LLM API Tokens (Gemini Flash)	\$0 (Free tier allowance) or <\$5	\$5 – \$15 / month	Flash-class pricing (\$0.075 / 1M input tokens); bounded 2,300 tokens per turn.
Observability & Logging	\$0 – \$15 / month	\$5 – \$25 / month	Cloud Logging exclusion filters scrub raw journal strings; retains telemetry only.
Security Testing & CI/CD	\$5 – \$15 / month	\$10 – \$25 / month	Adversarial suites run on GitHub Actions using local mock models and synthetic runs.
Total Recurring Monthly	\$10 – \$65 / month	\$55 – \$160 / month	Highly predictable unit cost of <\$0.30 per monthly active user.

8. Technical Architect Forum Presentation Guide

When presenting this solution to executive architects and engineering leadership, structure the live demonstration around three core technical pillars:

Pillar 1: Radically Accelerated Time-to-Market

Highlight how the combination of **Firestore Auth** and **Gemini API** collapses traditional backend plumbing. Instead of writing custom user tables, password-hashing daemons, and hosting private LLM weights on expensive GPU clusters, the system was implemented using standard TypeScript server functions and declarative rules. What historically took a 6-person team 4 months is now achievable by a single engineer in less than 5 days.

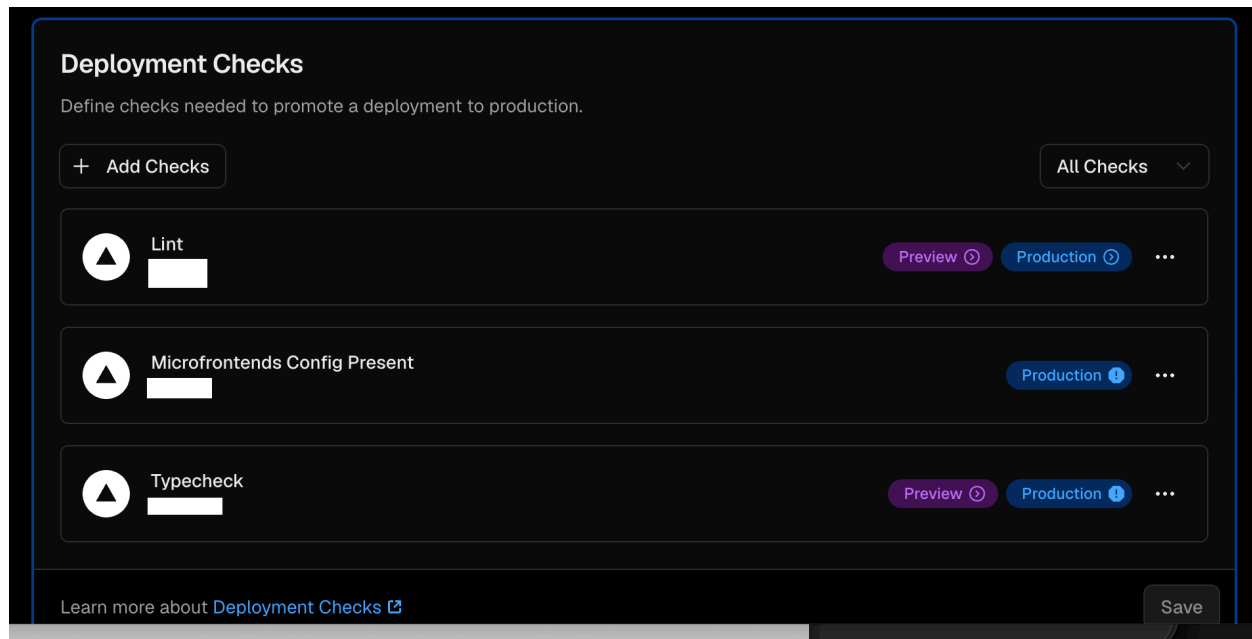
Pillar 2: Zero-Trust Tenant Isolation by Construction

Showcase the declarative security model. Demonstrate how cross-user exfiltration is physically impossible because Firestore enforces `request.auth.uid == userId` at the storage tier. Even if an attacker manipulates client-side JavaScript or attempts prompt injection to extract another user's journal IDs, the database engine drops the request at the wire level.

Pillar 3: Architectural Cost Predictability

Address the primary executive fear regarding GenAI: unpredictable operational expenses. Show how the 3-entry sliding window, the 300-word rolling summary, and the 300-token completion cap bound every interaction deterministically. The resulting workload scales linearly at pennies per active user without unexpected cloud cost spikes.

Deployment to Cloud Run From Github to Docker Managed Service



Similar to Below Could Run deployment can be done

```
Aug 21. project id : project-6ea80e7c-23f3-4e4c-817
gcloud config set project $GOOGLE_CLOUD_PROJECT
gcloud config set run/region $REGION
```

```
GOOGLE_CLOUD_PROJECT=project-6ea80e7c-23f3-4e4c-817
REGION=asea-southeast1
```

```
gcloud artifacts repositories describe my-repo --location= asea-southeast1
```

```
gcloud artifacts repositories list --project=Pproject-6ea80e7c-23f3-4e4c-817 \
--location=all
```

```
gcloud config set project $GOOGLE_CLOUD_PROJECT
gcloud config set run/region $REGION
```

```
gcloud beta run deploy Personal-Journal \
  --source=. \
  --region=$REGION \
  --sandbox-launcher \
  --max-instances=1 \
  --session-affinity \
  --allow-unauthenticated \
  --no-cpu-throttling \
  --labels dev-tutorial=codelab-cloud-run-personal-agent-coffee-shop \
  --set-env-vars
GOOGLE_GENAI_USE_VERTEXAI=1,GOOGLE_CLOUD_PROJECT=$GOOGLE_
CLOUD_PROJECT,GOOGLE_CLOUD_LOCATION=global,SPREADSHEET_ID=$
SPREADSHEET_ID \
  --service-account $SERVICE_ACCOUNT_ADDRESS
```

Data retention

Deployment Retention Policy

Define a retention period for all Deployments for this Project. Deployments will be deleted after the defined time.

Canceled Deployments

1 week

▼

Errored Deployments

1 day

▼

Pre-Production Deployments

1 day

▼

Production Deployments

30 days

▼